

JAVA

Introduzione a java



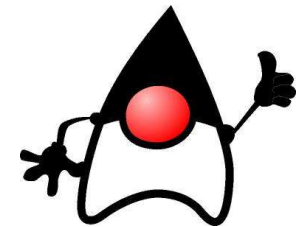
Storia: nascita di Oak

- ◆ Nel 1991, la Sun Microsystem si propose di creare una tecnologia per l'industria dell'elettronica (televisori, piccoli dispositivi domestici,...)
 - sviluppare un ambiente software platform-independent da installare su piccoli e portabili "articoli elettronici"
- ◆ Il **Green Team**, guidato da Gosling, creò un'estensione del compilatore C++, ma non rispettò le aspettative
- ◆ Così nel **1991** nacque **Oak**, un linguaggio
 - Object oriented
 - platform-independent
 - robusto e sicuro



Storia: *7

- ◆ Nel 1992 il Green Team rilasciò “*7 (**Star Seven**)” che era basato su 4 tecnologie sviluppate dal team:
 - Il linguaggio Oak
 - GreenOS, un sistema operativo
 - Una interfaccia utente
 - Il dispositivo stesso
- ◆ Essi utilizzarono nella presentazione un personaggio (**Duke**), che guidava l'utente attraverso questo mondo virtuale.
- ◆ A questo punto il team passò dal mercato dell'**elettronica** a quello della **tv on-demand**, ma dopo alcuni anni dovettero abbandonare il settore.



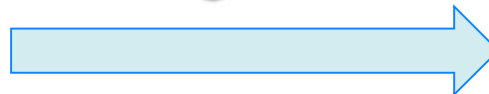
Storia: nascita di Java

- ◆ Nel 1994, dopo il fallimento, Oak intraprese la direzione del web
- ◆ Nel gennaio 1995 fu rinominato **Java**
- ◆ Nello stesso anno, Van Hoff scrisse un compilatore dedicato
- ◆ Venne inoltre progettato un nuovo browser, scritto in java, chiamato **HotJava**
- ◆ Il 23 **Maggio 1995**, in occasione del SunWorld '95, Sun Microsystem presentò Java.
- ◆ Nello stesso anno Netscape incorporò Java nel suo browser:
 - il successo è grande perché sfrutta le nuove esigenze del mercato del software, in particolare le tecnologie di rete e internet



Storia: Java e Oracle

- ◆ Nel **2010** la Sun Microsystems è stata assorbita da Oracle
- ◆ In seguito Oracle ha anche assorbito numerose società produttrici di tecnologie java-oriented:
 - Bea,
 - Mysql



ORACLE®

Principali caratteristiche di Java

- ♦ **Object Oriented**
- ♦ **Semplice (evoluzione del C)**
- ♦ **Portabile → Indipendente dalla piattaforma**
- ♦ **Robusto**
- ♦ **Concorrente (multithreading)**
- ♦ **Sicuro**

Infine gratuito e open source

Object Oriented

- ◆ Object Oriented è un paradigma di programmazione (non un linguaggio) fondato sui concetti di:
 - Incapsulamento
 - Ereditarietà
 - Polimorfismo
 - ◆ Principali linguaggi OO sono:
 - C++
 - Objective C
 - **Java**
 - Perl 5
 - Python
 - PHP 5
 - Java script (*debolmente OO*)
 - C#
 - Scala 2004
 - Kotlin 2011
 - Swift 2014
- Coetanei di Java
- ➡ Più recenti

Semplice

- ♦ Sintassi simile a C e C++
- ♦ Completamente "tipizzato"
- ♦ Elimina i costrutti più "pericolosi" di C e C++
 - aritmetica dei puntatori
 - (de)allocazione esplicita della memoria
- ♦ Funzionalità di garbage collector (GC) automatica

Svantaggi:

- ♦ Carente di funzioni a basso livello:
 - in questi casi si utilizza JNI, Java Native Interface

Indipendente dalla piattaforma

Un programma Java si **scrive una volta** sola e deve **girare su tutte** le piattaforme che hanno una macchina virtuale.

```
public class Studente  
{  
...  
...  
}
```



JVM per Linux



Piattaforma Linux

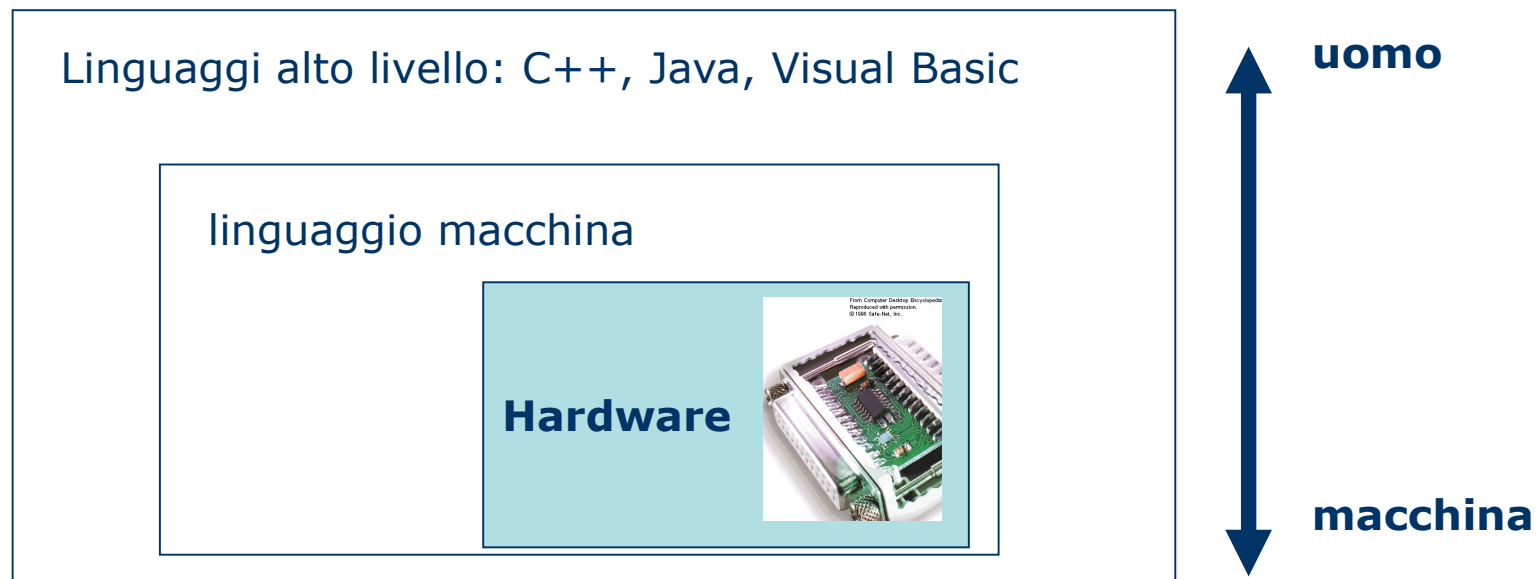
JVM per Windows



Piattaforma Windows

Linguaggi di programmazione

- ♦ Un linguaggio di programmazione consente di impartire istruzioni alla macchina
- ♦ L'unico linguaggio "comprensibile" alla macchina è il **linguaggio macchina**
- ♦ I **linguaggi di alto livello** servono per scrivere istruzioni ad alto livello, che devono essere poi **tradotte** in linguaggio macchina



Linguaggi compilati

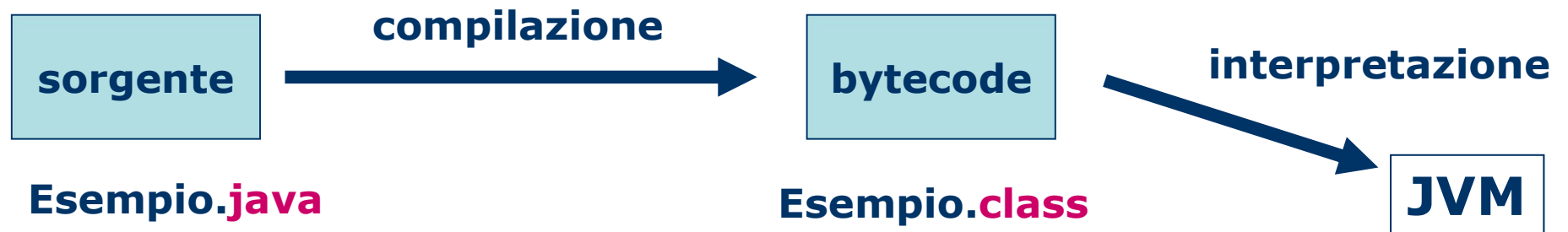
- ♦ I linguaggi compilati sono quelli che usano il programma **compilatore** per:
 - Rilevare eventuali errori di sintassi e di semantica
 - Tradurre il codice sorgente in codice macchina
- ♦ Esempio di linguaggio compilato è il C



- ♦ Esiste un compilatore per ogni piattaforma
 - un file .exe è eseguibile solo sulla piattaforma per la quale è stato compilato

Linguaggi interpretati

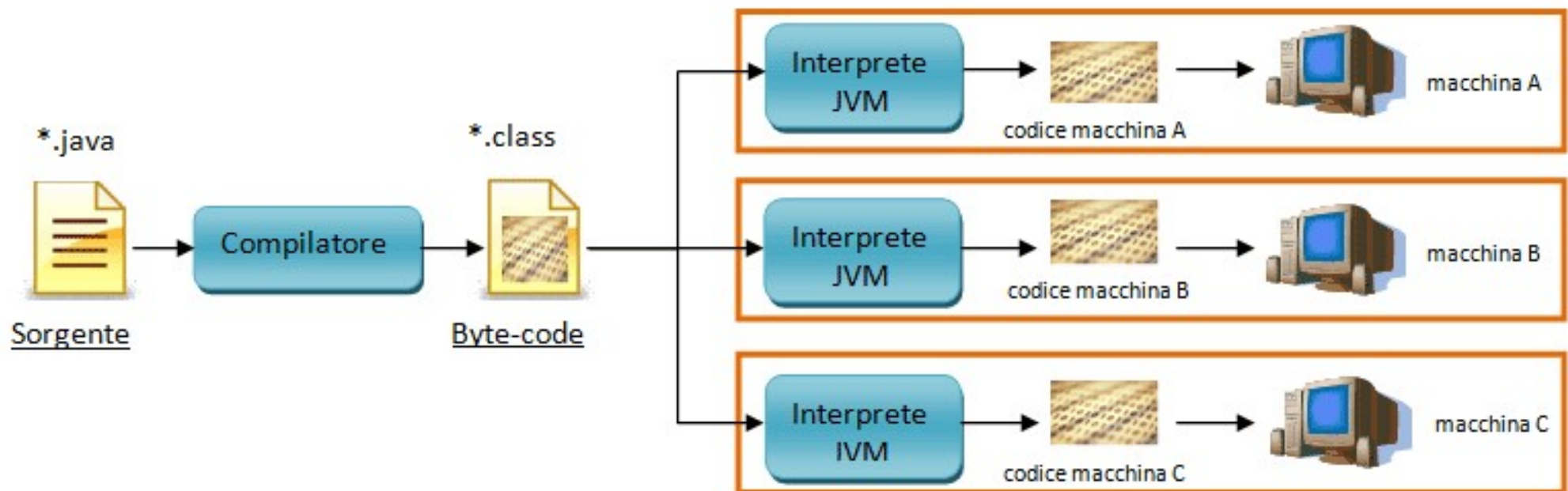
- ♦ Un linguaggio interpretato usa il **compilatore** per
 - Rilevare errori (sintassi e semantica)
 - Tradurre il codice sorgente in **bytecode**, linguaggio macchina virtuale da interpretare successivamente
- ♦ Un linguaggio interpretato prevede l'uso di un 2° programma: **l'interprete**



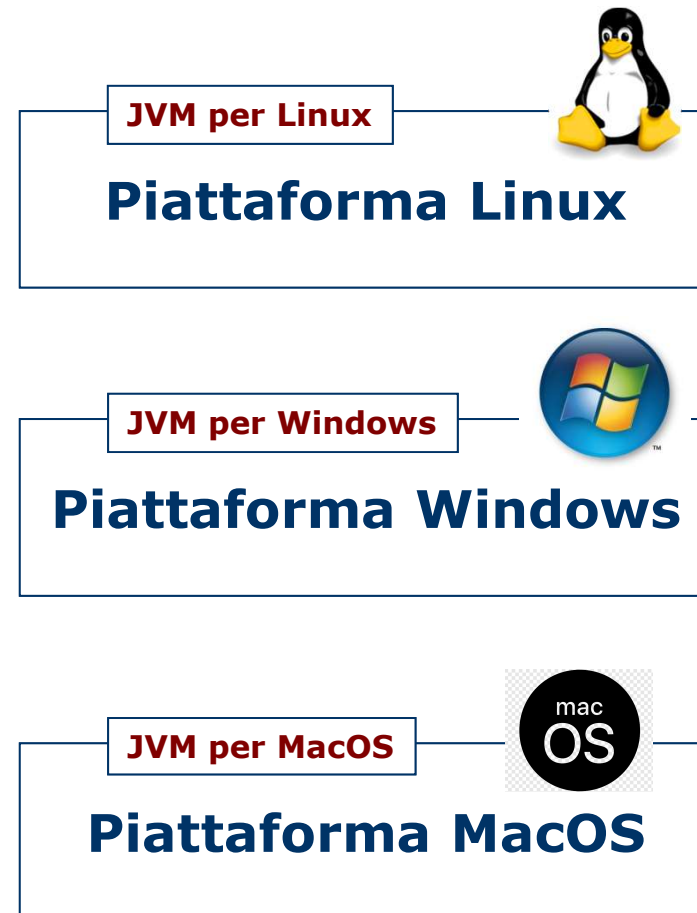
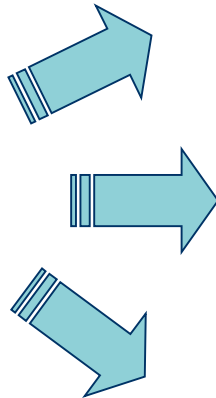
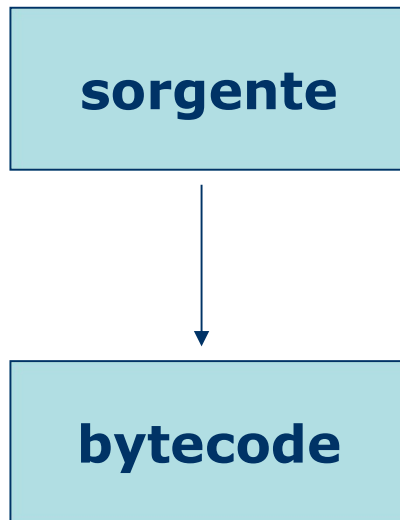
- ♦ Il bytecode viene eseguito istruzione per istruzione e non viene mai creato l'eseguibile!

La JVM

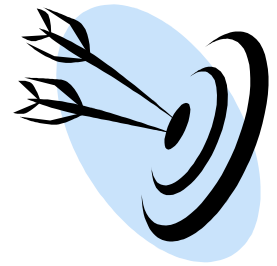
- ♦ La **JVM** è la *specifica* di un software che interpreta il bytecode java
- ♦ Le varie implementazioni consentono la traduzione del codice java in codice macchina specifico.



Indipendenza dalla piattaforma



...il momento giusto!



- ♦ I linguaggi interpretati esistevano anche prima di java, ma il meccanismo dell'interpretazione del codice divenne una necessità solo intorno agli anni 90 (con il boom di internet)

Robusto (affidabile)

- ◆ Rilevamento errori e Typechecking a tempo di compilazione e di esecuzione
- ◆ Mascheramento dei puntatori all'utente
- ◆ Gestione delle eccezioni da parte dell'utente
- ◆ Gestione della memoria (allocazione & GC)

Concorrente

- ◆ Come il sistema operativo è in grado di gestire contemporaneamente più programmi (***Multitasking***), in Java è possibile mandare in esecuzione parti diverse di uno stesso programma contemporaneamente (***Multithreading***)
- ◆ Nella libreria della Sun sono disponibili strumenti (API) che consentono di sviluppare facilmente programmi multithreading

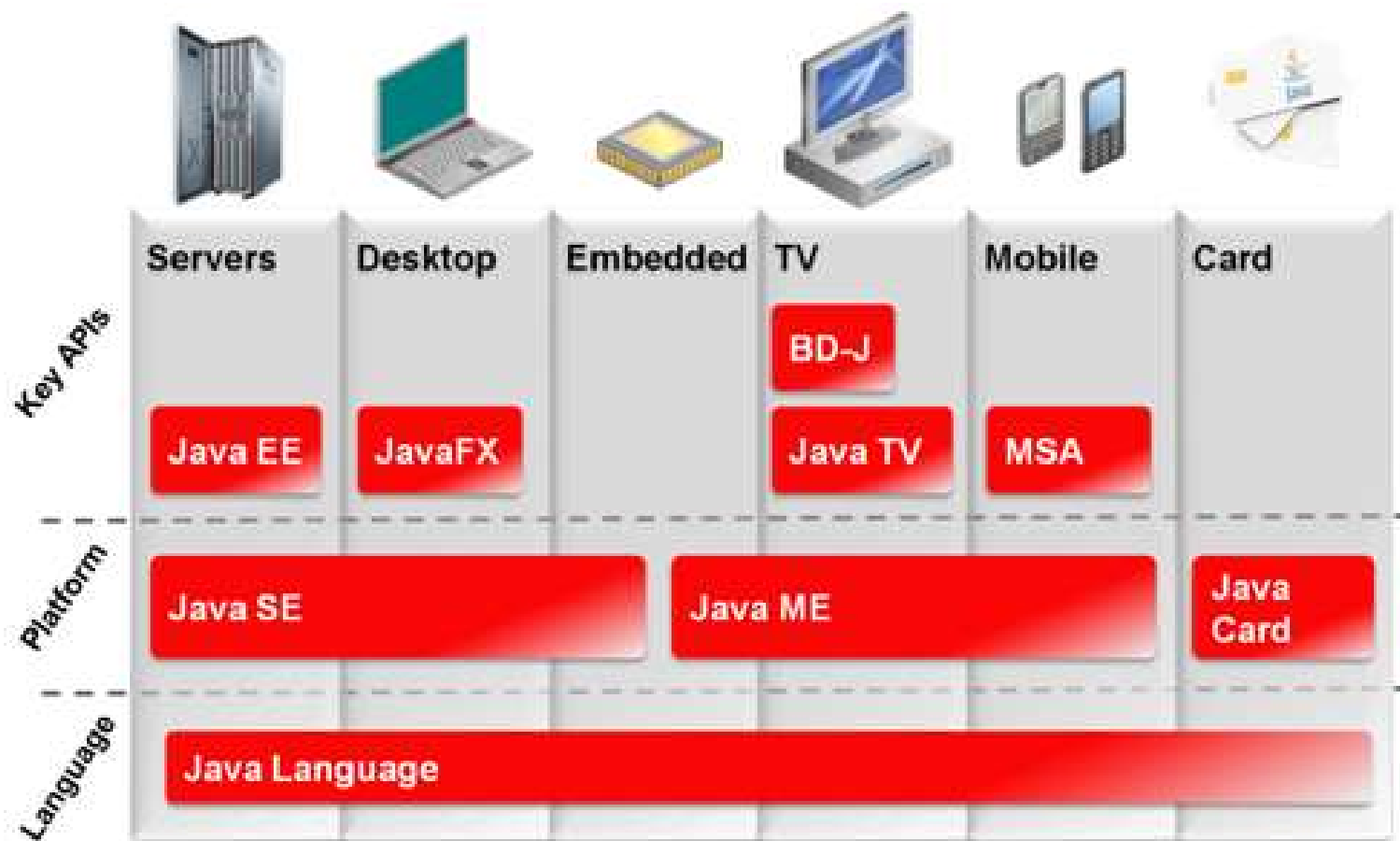
Sicuro

- ♦ Spesso informazioni di vitale importanza risiedono su sistemi informatici e sovente viaggiano in rete per essere fruibili dai più svariati servizi.
- ♦ Per questo uno dei punti di forza di Java è sicuramente la sicurezza
- ♦ Scelte architetturali:
 - Assenza dei puntatori (che non permette di manipolare locazioni di memoria in modo arbitrario)
 - No overrun dello stack a runtime
- ♦ API disponibili:
 - JCE (Java Cryptography Extension)
 - JSSE (Java Secure Socket Extension)
 - JAAS (Java Authentication and Authorization Service)

Java: linguaggio e tecnologia

- ♦ Il linguaggio Java è il linguaggio di programmazione più usato del pianeta
- ♦ Con il termine "Java" ci si riferisce sia al linguaggio di programmazione sia alla tecnologia
- ♦ Tecnologia Java indica una famiglia di prodotti di cui il linguaggio java è solo una parte e che include "sotto-tecnologie" affermate ambito software e hardware
- ♦ Segue tabella descrittiva →

Java: linguaggio e tecnologia



Utilizzare Java

Java Development Kit



Che cosa serve?

- ♦ Dal sito della **Oracle** è possibile scaricare il Kit base per lavorare in Java che comprende:
 - Il compilatore
 - L'interprete java (JVM per la piattaforma prescelta)
 - La libreria standard
 - altri tools utili
- ♦ Il kit si chiama **JDK – java development kit**
 - dipende dalla piattaforma → Linux, Windows, Mac OS
 - ha una versione
→ 8 (2014), 11 (2018), 13 (2019), 15 (2020), 19 (2022)
- ♦ Esistono 2 kit principali
 - **Java SE** → JDK con Standard Edition
 - **Java EE** → JDK + supporto applicazioni distribuite

Ambienti

JDK

`javac, jar, debugging tools,
javap`

JRE

`java, javaw, libraries,
rt.jar`

JVM

Just In Time
Compiler (JIT)

L'ambiente di sviluppo
contiene l'ambiente di
esecuzione che contiene
la JVM

JDK serve per
sviluppare, JRE è
sufficiente per eseguire

Variabili d'ambiente

- ♦ Sotto Pannello di controllo → sistema → avanzate → variabili d'ambiente
- ♦ **PATH = jdk_path\bin**
I programmi java si possono compilare e lanciare da tutte le cartelle.
- ♦ **CLASSPATH=.;jdk_path\lib**
Si specifica dove cercare le classi coinvolte nei programmi che si vanno a scrivere (comprese quelle della libreria)
Dove **jdk_path** è il path della JDK

Esempio:

```
PATH = C:\Programmi\Java\jdk1.8.0_91\bin
```

```
CLASSPATH = .; C:\Programmi\Java\jdk1.8.0_91\lib
```

Primo programma

Hello, World_

```
public class HelloWorld
{
    public static void main(String[] args)
    {
        System.out.println("Hello world!");
    }
}
```

Salvare il file come **HelloWorld.java**

Lanciare

<code>javac HelloWorld.java</code>	(compilazione)
<code>java HelloWorld</code>	(esecuzione)

Oppure con l'opzione `-cp` per indicare il classpath

Cioè il path fino al file.class o al pacchetto se esiste



APPENDICE

LE VERSIONI DI JAVA

Versioni 5/6

- ◆ Storico delle versioni:

- **5 Tiger - 2004**

Generics, for each loop, enum, Varargs,
Autoboxing/unboxing e le Annotations.



- **6 Mustang - 2006**

web service, JDBC 4



- ◆ Le versioni precedenti alla 1.5 sono ancora presenti su vecchi progetti → sconsigliato utilizzo!
- ◆ Dalla vers.5 in su, si parla di **Java 5** e non più 1.5

Versione 7

- ◆ Nel luglio 2011 nasce **Dolphin**, la versione 7
- ◆ Principali novità del linguaggio:
 - switch sulle stringhe
 - catch multipli
 - try che gestiscono la chiusura automatica
 - notazione binaria dei numeri interi
 - L'underscore sui literal numerici
 - Deduzione automatica del tipo generic in fase costruzione
- ◆ Principali novità di API:
 - Framework Fork/Join → gestione thread
 - JDBC 4.1 → accesso ai dati
 - NIO 2.0 → nuova libreria I/O
 - Nimbus: nuovo look 'n feel per UI



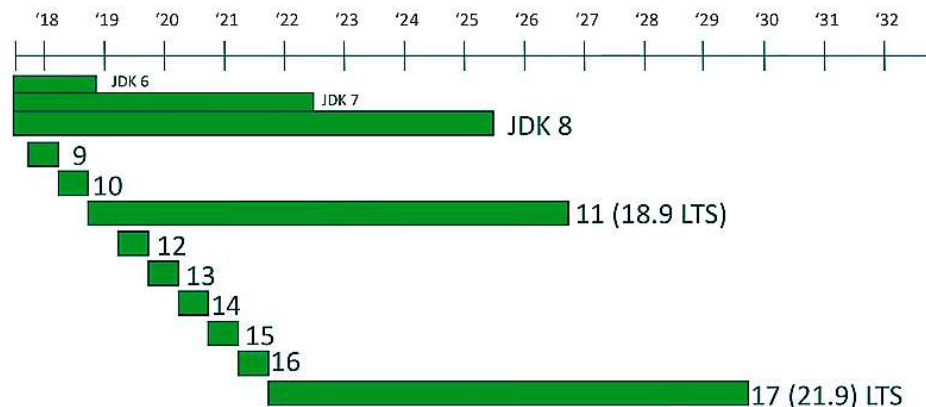
Versione 8

- ♦ Nel marzo 2014 nasce java 8, la prima versione **Oracle made** che sarà anche l'ultima totalmente gratuita
- ♦ Principali novità del linguaggio:
 - Espressioni lambda → evoluzione delle funzioni anonime
 - Metodi di default per le interfacce
- ♦ Novità nelle API:
 - java.time → gestire data e ora
 - Nuovi stream
- ♦ Altre caratteristiche:
 - Nashorn → engine della JDK per eseguire Javascript sulla JVM
 - Profili compatti → sottoinsiemi della libreria di sistema
 - Garbage Collector → migliorato in performance e sicurezza

Da java 9 in poi....

- ◆ Vengono introdotte:
 - le **LTS** (Long term Support)
versioni per cui viene garantito supporto e stabilità per 3 anni
(queste danno affidabilità agli ambienti di produzione)
 - le **feature release**
versioni con nuove funzionalità e migliorie (rilasciate affinché gli sviluppatori possano sperimentare nuove features)

Nuovo modello di rilascio delle versioni Java a partire da JDK9



Java 8 pur **non** essendo un LTS version avrà un **supporto fino al 2025**, quindi resta una valida versione **gratuita** ancora molto sfruttata!

Versione 11

- ♦ Le versioni 9 e 10 sono **features release** che apportano alcune migliorie, fino quando, nel marzo 2018, viene rilasciata la versione 11, che risulta la prima versione **LTS** ed a pagamento (come tutte le versioni LTS)
- ♦ Principali novità del linguaggio/API:
 - Inferenza del tipo con la keyword **var**
 - Metodi nuovi per Stream e String
- ♦ Principali novità:
 - migliore efficienza in termini di crittografia e sicurezza
 - introduzione di un Z-Garbage Collector più efficiente
 - Razionalizzazione dei pacchetti già deprecati in java 9
 - Rimozione di alcuni vecchi moduli (CORBA, Java EE) → perdita della retrocompatibilità da java 8 indietro!!!